LAPORAN TUGAS

Klasifikasi Sentimen Tweet Maskapai Penerbangan

Nama: Salsabila Nurul Fitri

Dataset: DatasetPMA.csv

1. Dataset dan Pembagian Data

Dataset DatasetPMA.csv berisi 14.640 data dan 15 kolom. Kolom text digunakan sebagai input dan airline_sentiment sebagai target.

Data	Jumlah
Total	14.640
Training	11.712 (80%)
Testing	2.928 (20%)

Dataset sebanyak 14.640 data berhasil dibagi menjadi 11.712 data training (80%) dan 2.928 data testing (20%). Pembagian dilakukan menggunakan stratifikasi berdasarkan variabel target airline_sentiment, sehingga proporsi kelas negative, neutral, dan positive tetap relatif terjaga pada kedua subset. Penggunaan random_state=42 memastikan pembagian data dapat direproduksi. Meskipun demikian, distribusi kelas masih menunjukkan ketidakseimbangan, dengan kelas negative memiliki jumlah data lebih banyak dibandingkan kelas neutral dan positive. Kondisi ini bukan merupakan kesalahan pembagian data, tetapi perlu diperhatikan dalam evaluasi performa model sehingga digunakan metrik macro-averaged seperti Macro-F1.

2. TF-IDF

Teks direpresentasikan menggunakan TF-IDF dengan max_features=20.000, ngram_range=(1,2), dan min_df=2. TF-IDF di-fit hanya pada training, kemudian data testing hanya ditransformasi.

Data	Shape
X_train	(11.712, 20.000)
X_test	(2.928, 20.000)

Catatan :TF-IDF hanya di-fit pada data training, sedangkan data testing hanya ditransformasi menggunakan vocabulary yang diperoleh dari training. Prosedur ini dilakukan untuk mencegah data leakage. Penggunaan ngram_range=(1,2) memungkinkan model mempertimbangkan unigram dan bigram, sedangkan min_df=2 menghilangkan term yang hanya muncul pada satu dokumen. Parameter max_features=20.000 membatasi jumlah fitur yang digunakan. Hasil representasi memiliki dimensi yang sama pada training dan testing, yaitu masing-masing 11.712 × 20.000 dan 2.928 × 20.000.

4. Baseline

Baseline menggunakan TF-IDF tanpa normalisasi tambahan. Ketiga algoritma, yaitu Logistic Regression, Linear SVM, dan Naive Bayes, diuji menggunakan representasi fitur yang sama. Hasil pada tahap ini digunakan sebagai **acuan (baseline/reference)** untuk membandingkan perubahan performa pada eksperimen selanjutnya.

Model	**Accuracy**	**Precision**	**Recall**	**Macro-F1**
Logistic Regression	0.8019	0.7937	0.6808	0.7203
Linear SVM	0.8043	0.7671	0.7168	0.7378
Naive Bayes	0.7135	0.7933	0.4951	0.5263

Catatan: Berdasarkan hasil baseline, performa ketiga algoritma menunjukkan perbedaan. Linear SVM memperoleh Accuracy sebesar 0,8043 dan Macro-F1 sebesar 0,7378, Logistic Regression memperoleh Accuracy sebesar 0,8019 dan Macro-F1 sebesar 0,7203, sedangkan Naive Bayes memperoleh Accuracy sebesar 0,7135 dan Macro-F1 sebesar 0,5263. Hasil tersebut **belum digunakan untuk menetapkan model akhir**, melainkan menjadi titik pembanding untuk melihat perubahan performa setelah penerapan normalisasi, PCA, regularisasi, dan feature selection.

5. Normalisasi

Normalisasi dilakukan menggunakan **MaxAbsScaler** karena sesuai untuk representasi TF-IDF yang bersifat sparse. MaxAbsScaler melakukan penskalaan setiap fitur berdasarkan nilai absolut maksimumnya tanpa mengubah nilai nol pada matriks sparse.

Secara matematis, transformasi MaxAbsScaler dapat dituliskan sebagai:

$$x'ij \quad = \frac{x_{ij}}{\max\ (|x_j|)}$$

Dengan x_{ij} merupakan nilai fitur ke-j pada data ke-i, $x'ij$ merupakan nilai setelah normalisasi, dan max $(|x_j|)$ merupakan nilai absolut maksimum dari fitur ke-j

Hasil eksperimen setelah normalisasi adalah sebagai berikut :

Model	**Accuracy**	**Precision**	**Recall**	**Macro-F1**
Logistic Regression	0.8029	0.7670	0.7075	0.7314
Linear SVM	0.7736	0.7218	0.6976	0.7080
Naive Bayes	0.7900	0.7512	0.6817	0.7085

Catatan :

Hasil normalisasi menunjukkan bahwa pengaruh penskalaan fitur berbeda pada masing-masing algoritma. **Logistic Regression** mengalami peningkatan Macro-F1 dari **0.7203 menjadi 0.7314**, sedangkan **Linear SVM** mengalami penurunan dari **0.7378 menjadi 0.7080**. **Naive Bayes** mengalami peningkatan Macro-F1 yang cukup besar, yaitu dari **0.5263 menjadi 0.7085**.

Perubahan tersebut dapat dikaitkan dengan karakteristik masing-masing algoritma. Pada Logistic Regression, prediksi didasarkan pada kombinasi linear fitur:

$$P\ (\ y = k|x) = \frac{e^{\beta_k^T x}}{\sum_{j=1}^{K} e^{\beta_j^T x}}$$

Sehingga perubahan skala fitur dapat memengaruhi nilai kombinasi linear $\beta_k^T x$ dan probabilitas kelas yang dihasilkan.pada linear SVM, fungsi keputusan juga bergantung pada kombinasi linear fitur :

$$f\ (x) = \ w^T x + b$$

Sehingga perubahan skala fitur dapat memengaruhi nilai fungsi keputusan dan posisi data relatif terhadap batas keputusan (*hyperplane*). Pada Naive Bayes, perubahan representasi nilai fitur juga dapat memengaruhi estimasi probabilitas fitur terhadap masing-masing kelas. Oleh karena itu, perubahan skala fitur dapat menghasilkan perubahan distribusi prediksi.

Dalam eksperimen ini, normalisasi tidak menghasilkan peningkatan performa yang seragam pada ketiga algoritma. Logistic Regression dan Naive Bayes mengalami peningkatan Macro-F1, sedangkan Linear SVM mengalami penurunan. Perbedaan tersebut menjadi hasil eksperimen yang selanjutnya dibandingkan dengan tahap **PCA, regularisasi, dan feature selection**.

6. PCA

Principal Component Analysis (PCA) diterapkan setelah tahap normalisasi untuk melakukan reduksi dimensi pada fitur TF-IDF. PCA diuji menggunakan **100, 300, dan 500 komponen**. Tujuan tahap ini adalah mengurangi jumlah dimensi fitur dengan mempertahankan informasi atau variasi data sebanyak mungkin dalam sejumlah komponen utama.

Secara matematis, PCA membentuk komponen utama berdasarkan kombinasi linear dari fitur awal:

$$Z = XW$$

Dengan :

X = matrix fitur setelah normalisasi,

W = matriks komponen utama

Z = representasi data dalam ruang berdimensi lebih rendah

Komponen utama ditentukan berdasarkan arah yang memiliki varians terbesar. Secara umum, arah komponen utama dapat diperoleh dari eigenvector matriks kovarians:

$$\sum w_k = \lambda_k w_k$$

Dengan λkmerupakan eigenvalue yang menunjukkan besar varians yang dijelaskan oleh komponen ke-k, sedangkan w_k merupakan eigenvector yang menentukan arah komponen tersebut.

Dengan demikian, semakin banyak komponen yang digunakan, semakin banyak informasi dari ruang fitur awal yang dipertahankan, tetapi dimensi data juga menjadi lebih besar.

Hasil transformasi PCA menghasilkan ukuran data sebagai berikut:

Komponen	**Training**	**Testing**
100	(11.712, 100)	(2.928, 100)
300	(11.712, 300)	(2.928, 300)
500	(11.712,500)	(2.928, 500)

Hasil Eksperimen PCA

Komponen	**Model**	**Accuracy**	**Precision**	**Recall**	**Macro-F1**
100	Logistic Regression	.7503	.7024	.6209	.6494
100	Linear SVM	.7490	.7129	.6105	.6437
300	Logistic Regression	.7756	.7360	.6612	.6895
300	Linear SVM	.7753	.7386	.6595	.6890
500	Logistic Regression	.7883	.7438	.6801	.7053
500	Linear SVM	.7859	.7397	.6776	.7022
500	Gaussian Naive Bayes	.4071	.4557	.4782	.3817

Catatan :

Hasil menunjukkan bahwa peningkatan jumlah komponen dari **100 menjadi 300 dan 500** diikuti oleh peningkatan performa Logistic Regression dan Linear SVM. Pada Logistic Regression, Macro-F1 meningkat dari **0.6494** pada 100 komponen menjadi **0.6895** pada 300 komponen dan

0.7053 pada 500 komponen. Linear SVM menunjukkan pola yang serupa, yaitu Macro-F1 meningkat dari **0.6437**, menjadi **0.6890**, kemudian **0.7022**.

Hal tersebut dapat dijelaskan secara konseptual bahwa penggunaan jumlah komponen yang lebih banyak memungkinkan lebih banyak variasi atau informasi dari data awal dipertahankan. Ketika hanya 100 komponen digunakan, sebagian informasi dari ruang fitur awal direpresentasikan dalam dimensi yang lebih kecil. Penambahan menjadi 300 dan 500 komponen memberikan ruang representasi yang lebih besar sehingga pada eksperimen ini performa kedua model meningkat.

Namun, peningkatan jumlah komponen tidak secara otomatis menjamin peningkatan performa**.** PCA berorientasi pada pemaksimalan varians data, bukan secara langsung pada pemisahan kelas. Oleh karena itu, komponen dengan varians besar belum tentu merupakan komponen yang paling informatif untuk klasifikasi.

Catatan untuk Naïve Bayes

PCA dapat menghasilkan nilai fitur positif maupun negatif karena setiap komponan merupakan kombinasi linear dari fitur awal:

$$z_k = \sum_{j=1}^{d} w_{jk} x_j$$

Meskipun $x_j$bernilai non-negatif, koefisien w_{jk} dapat bernilai positif maupun negatif. Oleh karena itu, hasil transformasi PCA dapat mengandung nilai negatif.

Karena Multinomial Naive Bayes dirancang untuk fitur berupa hitungan atau nilai non-negatif, model tersebut tidak digunakan secara langsung pada representasi PCA. Untuk eksperimen PCA digunakan Gaussian Naive Bayes**,** yang memodelkan distribusi kontinu setiap fitur dengan distribusi Gaussian:

$$\boldsymbol{P\left(x_j \,|y = k = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right)\right.}$$

Hal ini membuat GaussianNB lebih sesuai dengan karakteristik fitur hasil PCA yang kontinu dan dapat bernilai negatif. Pada eksperimen yang tersedia, hasil GaussianNB baru diperoleh untuk

500 komponen**,** dengan Accuracy 0.4071 dan Macro-F1 0.3817. Hasil untuk GaussianNB pada 100 dan 300 komponen belum tersedia sehingga tidak digunakan atau diinterpretasikan dalam laporan.

7. Regularisasi

Regularisasi dilakukan pada data hasil normalisasi untuk menguji pengaruh tingkat regularisasi terhadap performa masing-masing algoritma. Pada Logistic Regression dan Linear SVM, parameter yang divariasikan adalah C, sedangkan pada Multinomial Naive Bayes digunakan α sebagai parameter smoothing.

7.1 Logistic Regression

Pada Logistic Regression, regularisasi digunakan untuk mengendalikan kompleksitas model. Secara umum, fungsi objektif dengan regularisasi L2 dapat dituliskan sebagai:

$$\min_{\beta}[-\sum_{i=1}^{n}\log P\,(y_i|x_i) + \frac{1}{2C}||\beta||_2^2]$$

Parameter C merupakan kebalikan dari kekuatan regularisasi. Dengan demikian:

- C kecil → regularisasi lebih kuat → bobot model lebih dibatasi.
- C besar → regularisasi lebih lemah → model lebih bebas menyesuaikan data training.

Hasil eksperimen:

C	Accuracy	Precision	Recall	Macro-F1
.01	.6732	.7625	.4231	.4224
.10	.7903	.7910	.6528	.6961
1	.8029	.7670	.7075	.7314
10	.7869	.7397	.7026	.7180
100	.7859	.7324	.7058	.7173

Catatan : Dalam rentang C yang diuji, nilai C =1 C =1 menghasilkan Macro-F1 tertinggi, yaitu 0.7314. Nilai C =0.01 C =0.01 menghasilkan Macro-F1 yang jauh lebih rendah, yaitu 0.4224**,** yang menunjukkan bahwa regularisasi yang sangat kuat pada konfigurasi ini berkaitan dengan penurunan performa klasifikasi. Setelah C =1, C =1, peningkatan C menjadi 10 dan 100 tidak meningkatkan Macro-F1.

7.2 Linear SVM

Pada Linear SVM, parameter C juga mengatur trade-off antara margin dan kesalahan klasifikasi. Secara umum, formulasi primal soft-margin SVM dapat dituliskan sebagai:

$$\min_{w,b,\varepsilon} \frac{1}{2} ||w||^2 + C \sum_{i=1}^{n} \varepsilon_i$$

Dengan $\varepsilon_i$merupakan variabel slack yang memungkinkan adanya kesalahan klasifikasi.

Secara konseptual :

- C kecil → penalti terhadap kesalahan lebih kecil dan regularisasi relatif lebih kuat.
- C besar → penalti terhadap kesalahan lebih besar dan model lebih menekankan klasifikasi data tarining.

Hasil eksperimen :

C	**Accuracy**	**Precision**	**Recall**	**Macro-F1**
.01	.7732	.7961	.6164	.6645
.10	.8040	.7676	.7135	.7359
1	.7736	.7218	.6976	.7080
10	.7561	.6924	.6794	.6849
100	.7497	.6775	.6722	.6746

Catatan : Dalam rentang parameter yang diuji, C=0.10C=0.10 menghasilkan Macro-F1 tertinggi sebesar **0.7359**. Peningkatan CC dari 0.10 menjadi 1, 10, dan 100 justru diikuti penurunan Macro-F1 pada data testing.

Beberapa konfigurasi juga menghasilkan **convergence warning** selama proses pelatihan. Oleh karena itu, hasil tersebut perlu dicatat sebagai bagian dari kondisi eksperimen. Warning konvergensi tidak secara otomatis berarti hasil tidak dapat digunakan, tetapi menunjukkan bahwa solver belum mencapai kriteria konvergensi yang ditentukan dalam proses optimasi.

7.3 Naive Bayes

Pada Multinomial Naive Bayes, parameter yang digunakan adalah α. Berbeda dari C pada Logistic Regression dan SVM, α pada MultinomialNB merupakan **parameter smoothing**, bukan parameter regularisasi dalam pengertian yang sama.

Probabilitas fitur dengan Laplace/Lidstone smoothing secara umum dapat dituliskan:

$$P\left(x_j \mid y = k\right) = \frac{N_{jk} + \alpha}{N_k + \alpha d}$$

Dengan :

N_{jk} =jumlah / akumulasi fitur j pada kelas k

$N_k =$ total nilai fitur j pada kelas k

d =jumlah fitru

α = parameter smooting.

Semakin besar α, semakin besar pengaruh smoothing terhadap estimasi probabilitas.

Hasil Eksperimen :

Alpha	Accuracy	Precision	Recall	Macro-F1
.01	.7753	.7225	.6618	.6854
.10	.7879	.7292	.7016	.7136
1	.7900	.7512	.6817	.7085
10	.6762	.7754	.4260	.4266
100	.6335	.7121	.3458	.2833

Catatan : Dalam nilai α yang diuji, α =0.10 menghasilkan Macro-F1 tertinggi sebesar **0.7136**. Peningkatan α menjadi 10 dan 100 diikuti penurunan performa yang cukup besar. Hal ini menunjukkan bahwa smoothing yang terlalu besar pada konfigurasi eksperimen ini dapat mengubah estimasi probabilitas fitur secara berlebihan.

8. Feature Selection

Feature selection menggunakan SelectKBest dengan chi2 pada data normalisasi. Nilai k yang diuji adalah 1.000, 3.000, 5.000, dan 10.000.

k	Model	Accuracy	Precision	Recall	Macro-F1
1.000	Logistic Regression	.7842	.7508	.6799	.7076
1.000	Linear SVM	.7790	.7411	.6782	.7034
1.000	Naive Bayes	.7510	.7237	.6133	.6464
3.000	Logistic Regression	.7988	.7624	.7022	.7268
3.000	Linear SVM	.7883	.7425	.6942	.7145
3.000	Naive Bayes	.7807	.7590	.6592	.6917
5.000	Logistic Regression	.8009	.7629	.7023	.7269
5.000	Linear SVM	.7831	.7372	.6905	.7101
5.000	Naive Bayes	.7780	.7400	.6622	.6899
10.000	Logistic Regression	.7995	.7622	.7016	.7259
10.000	Linear SVM	.7725	.7202	.6892	.7024
10.000	Naive Bayes	.7883	.7498	.6817	.7079

Jumlah fitur yang terlalu sedikit (1.000) menurunkan performa. Nilai k dengan Macro-F1 tertinggi dalam rentang pengujian berbeda antaralgoritma.

9. Catatan Penting untuk Laporan

- Ketiga algoritma diuji pada setiap skenario; tidak ada penghentian eksperimen setelah memilih satu algoritma.
- Jangan menyebut satu model sebagai model akhir hanya berdasarkan satu tahap karena regularisasi dan feature selection masih bagian dari eksperimen.
- PCA diterapkan setelah normalisasi.
- MultinomialNB digunakan pada data non-negatif seperti TF-IDF/hasil normalisasi. Untuk hasil PCA digunakan GaussianNB karena fitur PCA dapat bernilai negatif.

- Hasil GaussianNB yang tersedia untuk PCA adalah PCA 500. PCA 100 dan 300 perlu dihitung jika laporan mensyaratkan semua kombinasi lengkap.
- Pada regularisasi Linear SVM terdapat convergence warning pada sebagian konfigurasi. Catatan ini sebaiknya tetap dicantumkan.
- Macro-F1 digunakan sebagai metrik utama pembanding karena distribusi kelas sentimen tidak seimbang.
- Nilai yang belum dihitung tidak diisi dengan perkiraan.

10. Kesimpulan Sementara

Eksperimen menunjukkan bahwa preprocessing, reduksi dimensi, regularisasi, dan feature selection memberikan pengaruh yang berbeda terhadap ketiga algoritma. Baseline memberikan gambaran awal performa. Normalisasi mengubah performa secara berbeda pada setiap model. PCA berhasil mengurangi dimensi dari 20.000 fitur menjadi 100–500 komponen, tetapi hasil Logistic Regression dan Linear SVM pada PCA masih berada di bawah hasil normalisasi tanpa PCA. Regularisasi dan feature selection menunjukkan bahwa parameter dan jumlah fitur dapat memengaruhi performa.

Kesimpulan akhir sebaiknya dibuat setelah seluruh skenario yang diwajibkan selesai dan hasilnya dibandingkan secara konsisten.